🎯 Project: Inventory Management System

Overview:

To build an advanced Inventory Management System with multiple collection types (HashSet, TreeSet, LinkedList) ,sorting, searching and data structure optimization for effective inventory operations.

Project Structure:

Class	Function
Product	Product class containing the attributes of products and extends Comparable<Product>.
Transaction	Abstract class to store transaction details
AddTransaction	To store transactions related to adding a product.
UpdateTransaction	To store transactions related to updating a product quantity.
PriceComparator	To compare products based on price.
ValueComparator	To compare products based on inventory value.
NameComparator	To sort products based on name.
InventoryUtils	Contains all the utility functions.
InventoryManagementSystem	To implement all the utility functions.

Methods in InventoryUtils:

Method	Utility
addProduct (Product product	Adding a product to the database.
updateProduct(String sku, int quantity)	Update the quantity of an existing product.
viewproducts(int criteria)	View products in the desired sorted order.
searchProduct(String search)	Searching a product based on a given keyword
lowStockAlert()	Display products with stock quantity<=10.
showTransaction ()	Display the last five transactions

Collections Used:

Collection	Use
HashSet	To store the collection of products in the inventory.
LinkedList	To store the transaction history.
ArrayList	To store products with low stock quantity.
HashMap	To store the count of items and total value of items per category.

Code:

```
package Product123;
import java.util.*;
//Basic Product class that extends Comparable<Product>
public class Product implements Comparable<Product> {

		//Unique Id of product
	private String sku;

	//Name of Product
	private String name;

	//category
	private String category;

	//Price
	private double price;

	//Quantity
	private int quantity;

	//To store the last time product was updated
	private Date lastUpdated;

	//Parameterized constructor to initialize variables
	public Product(String sku,String name,String category,double price,int quantity) {
		this.sku=sku;
		this.name=name;
		this.category=category;
		this.price=price;
		this.quantity=quantity;
		this.lastUpdated=new Date();
	}

	//Overriding comapreTo method
	@Override
	public int compareTo(Product other) {
		return this.sku.compareTo(other.sku);
```

```
}

//Overriding equals method
@Override
public boolean equals(Object obj) {
    if(this == obj) return true;
    if(obj == null || getClass() != obj.getClass()) return false;
    Product product = (Product) obj;
    return sku.equals(product.sku);
}

//Getters
@Override
public int hashCode() {
        return sku.hashCode();
}
        public String getSku() {
                return sku;
        }
        public String getName() {
                return name;
        }
        public String getCategory() {
                return category;
        }
        public double getPrice() {
                return price;
        }
        public int getQuantity() {
                return quantity;
        }
        public Date getLastUpdated() {
                return lastUpdated;
        }

        //Setters for selected attributes
        public void setPrice(double price) {
                this.price = price;
                this.lastUpdated=new Date();
        }
        public void setQuantity(int quantity) {
                this.quantity = quantity;
                this.lastUpdated=new Date();
        }
        public double getInventoryValue() {
             return price * quantity;
          }

        //Display product details
        public void showdetails() {
                System.out.println("Id: "+sku);
                System.out.println("Name: "+name);
                System.out.println("Category: "+category);
                System.out.println("Price: "+price);
```

```
                System.out.println("Qunaity: "+quantity);
        }

}
```

```
package Product123;
//Class to store transaction details
abstract class Transaction {

        //type of transaction:ADD or UPDATE
    String type;
    Product p;

    //Parameterized constructor
    Transaction(String type,Product p){
        this.type=type;
        this.p=p;
    }

    //Getters and setters
        public String getType() {
            return type;
        }
        public void setType(String type) {
            this.type = type;
        }
        public Product getP() {
            return p;
        }
        public void setP(Product p) {
            this.p = p;
        }

        //abstract method to display details of transaction
        public abstract void displaydetails();

}
```

```
package Product123;
//Class to store transaction of type: ADD
public class AddTransaction extends Transaction {
    AddTransaction(String type,Product p){
        super(type,p);
    }
    @Override
    public void displaydetails() {
        System.out.print(type+" "+p.getSku()+" Quantity: "+p.getQuantity()+" at "+p.getLastUpdated()+"\n");
    }
}
```

```
package Product123;
import java.util.Comparator;
```

```
//Overriding comparator method to compare products based on name
public class NameComparator implements Comparator<Product>{
    @Override
    public int compare(Product p1,Product p2) {
            return (p1.getName()).compareTo(p2.getName());
    }
}
```

```
package Product123;
import java.util.*;
//Overriding comparator method to compare products based on price
public class PriceComparator implements Comparator<Product>{
    @Override
    public int compare(Product p1,Product p2) {
            return Double.compare(p1.getPrice(),p2.getPrice());
    }
}
```

```
package Product123;
import java.util.*;
//Overriding comparator method to compare products based on inventory value
public class ValueComparator implements Comparator<Product>{
    @Override
    public int compare(Product p1,Product p2) {
            return Double.compare(p1.getInventoryValue(),p2.getInventoryValue());
    }
}
```

```
package Product123;
import java.util.*;
/* CLass that contains are the data structures: HashSet, LinkedList, ArrayList, HashMap to store the products,
transactions,
* products with low stock and map to count the total items and total value per category respectively.it also contains all
* the utility methods to ADD,UPDATE,SEARCH,VIEW the products in sorted order and also view the previous
transactions*/
public class Inventoryutils {

    //Linked List to store transaction history
    List<Transaction> transactions=new LinkedList<>();

    //Set to store the products
    HashSet<Product> products=new HashSet<>();

    //List to store products with low stock
    List<Product> lowstock=new ArrayList<>();

    //HashMap to store the count of goods of each category
    HashMap<String,Integer> categorycount=new HashMap<>();

    //HashMap to store the sum of inventory value of goods of each category
    HashMap<String,Double> categorysum=new HashMap<>();

    //Method to add product to the database
```

```
public void addProduct(Product product) {
	products.add(product);
	//Add the transaction to the transaction database
	transactions.add(new AddTransaction("ADDED",product));
}

//Method to update the quantity of an existing product
public boolean updateProduct(String sku,int quantity) {
	Product p=null;
	for(Product product:products) {
		if(product.getSku().equals(sku)) {
			p=product;
		}
	}
	if(p==null) {
		System.out.println("No Products found!!!");
		return false;
	}

		int initial=p.getQuantity();
		p.setQuantity(quantity);

		//Add the transaction to the transaction database
		transactions.add(new UpdateTransaction("UPDATED",p,initial,quantity));
		if(p.getQuantity()==0) {
			products.remove(p);
		}
		return true;
}

//View the products in a sorted order
public void viewproducts(int criteria) {
	List<Product> productList = new ArrayList<>(products);
	if(productList.isEmpty()) {
		System.out.println("No details to show");
	}

	//Products sorted based on price
	if(criteria==1) {
		Collections.sort(productList,new PriceComparator());
	}

	//Products sorted based on inventory value
	else if(criteria==2) {
		Collections.sort(productList,new ValueComparator());
	}

	//Products sorted based on name
	else if(criteria==3) {
		Collections.sort(productList,new NameComparator());
	}
	System.out.printf("%-10s %-20s %-15s %-10s %-8s %-12s\n",
	 "SKU", "Name", "Category", "Price", "Qty", "Value");
   System.out.println("-".repeat(85));
```

```
    //Display the sorted products
    for(Product product : productList) {
            System.out.printf("%-10s %-20s %-15s ₹%-9.2f %-8d ₹%-11.2f\n",
            product.getSku(), product.getName(), product.getCategory(),
            product.getPrice(), product.getQuantity(), product.getInventoryValue());
       }

}

//Search a product based on keyword
public void searchProduct(String search) {
        List<Product> searchresults=new ArrayList<>();
        search=search.toLowerCase();

        //Checks whether the search keyword matches with product id,name or category
        for(Product product:products) {
                if((product.getSku().toLowerCase().contains(search))
                                ||(product.getName().toLowerCase().contains(search))
                                ||(product.getCategory().toLowerCase().contains(search))) {
                        searchresults.add(product);
                }
        }
        System.out.println("SEARCH RESULTS FOR '"+search+"':");
        if(searchresults.isEmpty()) {
                System.out.println("No details to show");
        }
        System.out.printf("%-10s %-20s %-15s %-10s %-8s %-12s\n",
        "SKU", "Name", "Category", "Price", "Qty", "Value");
   System.out.println("-".repeat(85));

    //Display the search results
    for(Product product : searchresults) {
            System.out.printf("%-10s %-20s %-15s ₹%-9.2f %-8d ₹%-11.2f\n",
            product.getSku(), product.getName(), product.getCategory(),
            product.getPrice(), product.getQuantity(), product.getInventoryValue());
         }

}

//Display products with stock quantity<=10
public void lowStockAlert() {
        for(Product p:products) {
                if(p.getQuantity()<10) {
                        lowstock.add(p);
                }
        }
        System.out.println("Low stock alert for Products with Product Ids: ");
        if(lowstock.isEmpty()) {
                System.out.println("No details to show");
        }
        for(Product product:lowstock) {
                System.out.println(product.getSku());
        }
        lowstock=new ArrayList<>();
```

```
    }


    //Get the last five transactions
    public void showTransaction() {
            int len=transactions.size();
            if(len>5) {
            for(int i=len-6;i<len;i++) {
                    transactions.get(i).displaydetails();
            }
            }

            //If the number of transactions is less than 5, the show all the transactions
            else {
                    for(Transaction t:transactions) {
                            t.displaydetails();
                            System.out.println();
                    }
            }
    }

    //Display the count and total sum of each category
    public void InventoryStatistic() {
      for(Product product:products) {
            String cat=product.getCategory().toLowerCase();
            if(categorycount.get(cat)==null) {
                    categorycount.put(cat,1);
                    categorysum.put(cat, product.getInventoryValue());
            }
            else {
                    categorycount.put(cat,categorycount.get(cat)+1);
                    categorysum.put(cat,categorysum.get(cat)+ product.getInventoryValue());
            }
      }
      System.out.println("Total Products: "+products.size());
      System.out.println("-".repeat(100));
      for(String cat:categorycount.keySet()) {
            System.out.println("TYPE: "+cat);
            System.out.println("Products: "+categorycount.get(cat));
            System.out.println("Total value: "+categorysum.get(cat));
            System.out.println("-".repeat(100));
      }
     }

}
```

```
package Product123;
import java.util.Scanner;
//Contains the main method to implement all the utility methods under Inventoryutils
public class InventoryManagementSystem {
    public static void main(String args[]) {

            //Scanner class to take input
            Scanner sc=new Scanner(System.in);
```

```
Inventoryutils iu=new Inventoryutils();
int choice=0;

//Menu driven interface to perform the desired task
while(choice!=8) {
	System.out.println("\n=== INVENTORY MANAGEMENT SYSTEM ===\r\n"
			+ "1. Add Product\r\n"
			+ "2. Update Quantity\r\n"
			+ "3. View Products (Sorted)\r\n"
			+ "4. Search Products\r\n"
			+ "5. Low Stock Alerts\r\n"
			+ "6. Transaction History\r\n"
			+ "7. Inventory Statistics\r\n"
			+ "8. Exit\r\n"
			+ "Enter your choice:");
	choice=sc.nextInt();

	//Implement add product functionality
	if(choice==1) {
		System.out.println("\n===ADD PRODUCT===\n");
		System.out.println(".".repeat(100));
		System.out.println("Enter Product Id:");
		sc.nextLine();
		String sku=sc.nextLine();
		System.out.println("Enter Product Name:");
		String name=sc.nextLine();
		System.out.println("Enter Product Category:");
		String type=sc.nextLine();
		System.out.println("Enter Price:");
		double price=sc.nextDouble();
		System.out.println("Enter Quantity: ");
		int quantity=sc.nextInt();
		Product product=new Product(sku,name,type,price,quantity);
		iu.addProduct(product);
		System.out.println("Product Added!!");
	}

	//Implement the update product functionality
	else if(choice==2) {
		System.out.println("\n===UPDATE QUANTITY===\n");
		System.out.println(".".repeat(100));
		System.out.println("Enter product Id:");
		sc.nextLine();
		String sku=sc.nextLine();
		System.out.println("Enter Quantity: ");
		int quantity=sc.nextInt();
		if(iu.updateProduct(sku, quantity)) {
			System.out.println("Updated Succesfully!!!");
		}
	}

	//View sorted products
	else if(choice==3) {
		System.out.println("\n===VIEW PRODUCT===\n");
```

```
                        System.out.println(".".repeat(100));
                        System.out.println("View Products sorted by\n1.Price\n2.Inventory Value\n3.Name\n");
                        int ch=sc.nextInt();
                        iu.viewproducts(ch);
                }

                //Searching a product
                else if(choice==4) {
                        System.out.println("\n===SEARCH PRODUCT===\n");
                        System.out.println(".".repeat(100));
                        System.out.println("Enter keyword:");
                        sc.nextLine();
                        String search=sc.nextLine();
                        iu.searchProduct(search);
                }

                //Showing products with low stock
                else if(choice==5) {
                        System.out.println("\n===LOW STOCK ALERTS===\n");
                        System.out.println(".".repeat(100));
                        iu.lowStockAlert();
                }

                //Showing the transaction history
                else if(choice==6) {
                        System.out.println("\n===LAST 5 TRASNACTIONS===\n");
                        System.out.println(".".repeat(100));
                        iu.showTransaction();
                }

                //Display inventory statistics
                else if(choice==7) {
                        System.out.println("\n===INVENTORY STATISTICS===\n");
                        System.out.println("-".repeat(100));
                        iu.InventoryStatistic();
                }
        }
        sc.close();
    }
}
```

Output:

```
=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
```

```
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
1

===ADD PRODUCT===

.....
Enter Product Id:
E101
Enter Product Name:
Headphone
Enter Product Category:
Electronics
Enter Price:
500
Enter Quantity:
30
Product Added!!

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
1

===ADD PRODUCT===

.....
Enter Product Id:
C101
Enter Product Name:
Shaving Cream
Enter Product Category:
Cosmetics
Enter Price:
200
Enter Quantity:
60
Product Added!!

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
```

```
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
1

===ADD PRODUCT===

.....
Enter Product Id:
E102
Enter Product Name:
Fridge
Enter Product Category:
Electronics
Enter Price:
20000
Enter Quantity:
12
Product Added!!

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
6

===LAST 5 TRASNACTIONS===

.....
ADDED E101 Quantity: 30 at Wed Oct 28 14:10:16 IST 2026

ADDED C101 Quantity: 60 at Wed Oct 28 14:10:35 IST 2026

ADDED E102 Quantity: 12 at Wed Oct 28 14:10:59 IST 2026


=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
1
```

```
===ADD PRODUCT===

.....
Enter Product Id:
C102
Enter Product Name:
Comb
Enter Product Category:
Cosmetics
Enter Price:
30
Enter Quantity:
40
Product Added!!

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
1

===ADD PRODUCT===

.....
Enter Product Id:
F101
Enter Product Name:
Cookies
Enter Product Category:
Eateries
Enter Price:
40
Enter Quantity:
25
Product Added!!

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
3
```

```
===VIEW PRODUCT===

.....
View Products sorted by
1.Price
2.Inventory Value
3.Name

1
SKU       Name              Category       Price      Qty      Value
-------------------------------------------------------------------------------------
C102      Comb              Cosmetics      ₹30.00     40       ₹1200.00
F101      Cookies           Eateries       ₹40.00     25       ₹1000.00
C101      Shaving Cream     Cosmetics      ₹200.00    60       ₹12000.00
E101      Headphone         Electronics    ₹500.00    30       ₹15000.00
E102      Fridge            Electronics    ₹20000.00  12       ₹240000.00

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
3

===VIEW PRODUCT===

.....
View Products sorted by
1.Price
2.Inventory Value
3.Name

2
SKU       Name              Category       Price      Qty      Value
-------------------------------------------------------------------------------------
F101      Cookies           Eateries       ₹40.00     25       ₹1000.00
C102      Comb              Cosmetics      ₹30.00     40       ₹1200.00
C101      Shaving Cream     Cosmetics      ₹200.00    60       ₹12000.00
E101      Headphone         Electronics    ₹500.00    30       ₹15000.00
E102      Fridge            Electronics    ₹20000.00  12       ₹240000.00

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
```

```
7. Inventory Statistics
8. Exit
Enter your choice:
3

===VIEW PRODUCT===

.....
View Products sorted by
1.Price
2.Inventory Value
3.Name

3
SKU       Name              Category       Price      Qty      Value
-------------------------------------------------------------------------------
C102      Comb              Cosmetics      ₹30.00     40       ₹1200.00
F101      Cookies           Eateries       ₹40.00     25       ₹1000.00
E102      Fridge            Electronics    ₹20000.00  12       ₹240000.00
E101      Headphone         Electronics    ₹500.00    30       ₹15000.00
C101      Shaving Cream     Cosmetics      ₹200.00    60       ₹12000.00

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
4

===SEARCH PRODUCT===

.....
Enter keyword:
Cos
SEARCH RESULTS FOR 'cos':
SKU       Name              Category       Price      Qty      Value
-------------------------------------------------------------------------------
C101      Shaving Cream     Cosmetics      ₹200.00    60       ₹12000.00
C102      Comb              Cosmetics      ₹30.00     40       ₹1200.00

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
```

```
Enter your choice:
2

===UPDATE QUANTITY===

.....
Enter product Id:
E102
Enter Quantity:
6
Updated Succesfully!!!

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
5

===LOW STOCK ALERTS===

.....
Low stock alert for Products with Product Ids:
E102

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
6

===LAST 5 TRASNACTIONS===

.....
ADDED E101 Quantity: 30 at Wed Oct 28 14:10:16 IST 2026
ADDED C101 Quantity: 60 at Wed Oct 28 14:10:35 IST 2026
ADDED E102 Quantity: 6 at Wed Oct 28 14:12:57 IST 2026
ADDED C102 Quantity: 40 at Wed Oct 28 14:11:29 IST 2026
ADDED F101 Quantity: 25 at Wed Oct 28 14:12:00 IST 2026
UPDATED E102 Quantity changed from 12 to 6 at Wed Oct 28 14:12:57 IST 2026

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
```

```
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
7

===INVENTORY STATISTICS===

--------------------------------------------------------------------------------------------------
Total Products: 5
--------------------------------------------------------------------------------------------------
TYPE: electronics
Products: 2
Total value: 135000.0
--------------------------------------------------------------------------------------------------
TYPE: eateries
Products: 1
Total value: 1000.0
--------------------------------------------------------------------------------------------------
TYPE: cosmetics
Products: 2
Total value: 13200.0
--------------------------------------------------------------------------------------------------

=== INVENTORY MANAGEMENT SYSTEM ===
1. Add Product
2. Update Quantity
3. View Products (Sorted)
4. Search Products
5. Low Stock Alerts
6. Transaction History
7. Inventory Statistics
8. Exit
Enter your choice:
8
```